Tim Pengajar IF2150

IF2150 – Rekayasa Perangkat Lunak

Design Model Evaluation

SEMESTER I TAHUN AJARAN 2025/2026

KNOWLEDGE & SOFTWARE ENGINEERING

Tujuan Pembelajaran

- Memahami konsep dan prinsip evaluasi hasil perancangan perangkat lunak.
- Menjelaskan metode klasik dan modern evaluasi desain (Pressman 2010, 2020).
- Melakukan simulasi evaluasi rancangan dan dokumentasi hasil review.

Konsep Evaluasi Desain

- Tahap Verification & Validation untuk menjamin rancangan sesuai kebutuhan.
- Dilakukan sebelum implementasi agar kesalahan dapat dideteksi dini.
- Menjamin kesesuaian desain dengan spesifikasi pengguna.

Verification vs Validation

- **Verification:** Apakah desain dibangun dengan benar?
- **Validation:** Apakah desain yang dibuat sesuai dengan kebutuhan pengguna?
- Keduanya penting untuk menjaga kualitas produk perangkat lunak.

1. Verification – "Are we building the product right?"

- Verification memastikan **proses dan hasil desain dilakukan dengan benar** sesuai spesifikasi teknis dan metode yang ditetapkan.
- Fokusnya adalah pada **kebenaran proses rekayasa**, bukan apakah produk tersebut sesuai dengan harapan pengguna.
- Contoh pada tahap desain:
  - Memeriksa apakah diagram UML sesuai standar.
  - Mengecek apakah relasi antar modul konsisten dengan arsitektur sistem.
  - Melakukan *peer review* atau *design inspection*.
- Tujuan: menjamin **keakuratan desain secara teknis**.

KNOWLEDGE & SOFTWARE ENGINEERING

2. Validation – "Are we building the right product?"

- Validation memastikan bahwa **desain yang dibuat benar-benar sesuai dengan kebutuhan pengguna atau stakeholder**.
- Fokusnya pada **kesesuaian hasil desain terhadap kebutuhan dan ekspektasi pengguna**.
- Contoh pada tahap desain:
  - Meninjau apakah antarmuka pengguna mudah digunakan.
  - Mengevaluasi alur sistem terhadap *user requirements*.
  - Menguji prototipe kepada calon pengguna (*usability testing*).
- Tujuan: menjamin **relevansi dan manfaat desain bagi pengguna akhir**.

KNOWLEDGE & SOFTWARE ENGINEERING

3. Hubungan Verification & Validation

- **Verification ≠ Validation**, tetapi keduanya **saling melengkapi**.
- Verification memastikan **proses teknis benar**, sedangkan Validation memastikan **arahnya benar**.
- Tanpa verification, sistem bisa "benar secara salah" (dibangun dengan cara salah).
- Tanpa validation, sistem bisa "salah secara benar" (dibangun dengan baik, tapi tidak sesuai kebutuhan).

Prinsip Evaluasi Desain (Pressman, 2010)

- Proses sistematis untuk memastikan desain memenuhi prinsip kualitas.
- Melibatkan usability review, design walkthrough, dan prototyping evaluation.
- Fokus pada konsistensi, efisiensi, dan kesesuaian dengan user goals.

Metode Evaluasi Klasik

- **Design Review**: Pemeriksaan sistematis arsitektur.
- **Walkthrough**: Simulasi rancangan oleh tim.
- **Peer Review**: Evaluasi sejawat terhadap kesalahan dini.
- **Prototyping Evaluation**: Pengujian rancangan antarmuka pengguna.

1. Design Review

- Merupakan **pemeriksaan sistematis terhadap arsitektur dan model desain**.
- Dilakukan oleh tim teknis atau komite review untuk menilai:
  - Kesesuaian desain dengan *requirement specification*.
  - Konsistensi antar diagram (misalnya DFD, UML, class diagram).
  - Kelengkapan modul dan keterhubungannya.
- Biasanya menghasilkan *review report* berisi temuan dan rekomendasi perbaikan.
- **Tujuan:** mendeteksi kesalahan logika atau inkonsistensi sebelum sistem diimplementasikan.

KNOWLEDGE & SOFTWARE ENGINEERING

2. Walkthrough

- Merupakan **simulasi rancangan secara bertahap oleh tim pengembang**.
- Tim "menelusuri" desain seperti pengguna menjalankan sistem untuk memverifikasi alur dan fungsionalitas.
- Fokusnya pada **alur logika dan interaksi antar komponen**.
- Biasanya dipandu oleh pembuat desain, sementara anggota tim lain memberi masukan dan kritik.
- **Tujuan:** menemukan kekurangan rancangan dari perspektif implementasi dan pengguna.

3. Peer Review

- Melibatkan **evaluasi sejawat (rekan satu tim)** terhadap hasil rancangan.
- Biasanya dilakukan informal, cepat, dan berulang selama proses desain.
- Reviewer fokus pada kualitas dokumen, kesesuaian konvensi desain, dan kesalahan teknis kecil.
- Contoh: rekan memeriksa UML diagram atau *pseudocode* untuk mendeteksi redundansi.
- **Tujuan:** menjaga kualitas hasil desain sejak awal dengan biaya rendah dan umpan balik cepat.

4. Prototyping Evaluation

- Menggunakan **model awal (prototype)** untuk menguji rancangan dengan pengguna.
- Cocok untuk mengevaluasi *user interface (UI)*, alur navigasi, dan kebutuhan pengguna yang belum pasti.
- Umpan balik dari pengguna digunakan untuk memperbaiki desain sebelum implementasi penuh.
- **Tujuan:** memastikan rancangan sistem sesuai dengan ekspektasi dan pengalaman pengguna (usability).

Kriteria Evaluasi

- Kesesuaian dengan requirement specification.
- Modularitas dan keterpaduan antar komponen.
- Maintainability dan fleksibilitas desain.
- Efisiensi algoritma dan kepatuhan standar (UML, ISO/IEC 25010).

1. Kesesuaian dengan Requirement Specification

- Desain harus **selaras dengan kebutuhan fungsional dan non-fungsional** yang telah ditetapkan pada tahap analisis.
- Semua *requirement* harus dapat ditelusuri (traceable) ke elemen desain.
- Contoh: setiap *use case* memiliki representasi di *class diagram* atau *sequence diagram*.
- *Tujuan:* memastikan bahwa tidak ada kebutuhan pengguna yang terlewat atau salah diimplementasikan dalam desain.

KNOWLEDGE & SOFTWARE ENGINEERING

2. Modularitas dan Keterpaduan Antar Komponen

- Desain yang baik memiliki **struktur modular**, yaitu sistem dipecah menjadi bagian-bagian (modul) yang independen namun saling berhubungan secara logis.
- Modularitas memudahkan pengembangan, pengujian, dan pemeliharaan.
- Evaluasi dilakukan dengan menilai:
  - **Cohesion** (keterpaduan internal dalam modul)
  - **Coupling** (ketergantungan antar modul)
- *Tujuan:* meningkatkan kejelasan arsitektur dan mengurangi kompleksitas sistem.

KNOWLEDGE & SOFTWARE ENGINEERING

3. Maintainability dan Fleksibilitas Desain

- Desain harus mudah dimodifikasi tanpa menimbulkan efek besar pada bagian lain sistem.
- Termasuk kemampuan untuk melakukan pembaruan, adaptasi teknologi, atau perluasan fungsi.
- Faktor yang dinilai:
  - Dokumentasi desain yang jelas
  - Penggunaan standar dan pola desain (*design patterns*)
- *Tujuan:* memastikan sistem dapat bertahan lama dan mudah dikembangkan di masa depan.

KNOWLEDGE & SOFTWARE ENGINEERING

4. Efisiensi Algoritma dan Kepatuhan terhadap Standar

- Evaluasi dilakukan terhadap **efisiensi logika algoritma** (waktu, memori, dan performa).
- Desain juga harus mengikuti **standar desain** seperti:
  - UML (Unified Modeling Language) untuk pemodelan sistem.
  - ISO/IEC 25010 untuk dimensi kualitas seperti *performance, usability, reliability,* dan *security*.
- *Tujuan:* menjamin desain sistem efektif, dapat diimplementasikan dengan benar, dan sesuai praktik industri.

KNOWLEDGE & SOFTWARE ENGINEERING

Evaluasi Desain Modern (Pressman & Maxim, 2020)

- Evaluasi dilakukan secara berkelanjutan dalam tiap iterasi pengembangan.
- Fokus pada atribut kualitas: performance, scalability, modifiability, dan security.
- Diterapkan melalui Agile iteration dan architectural review.

1. Evaluasi dilakukan secara berkelanjutan dalam tiap iterasi pengembangan

- Pada metode tradisional (*waterfall*), evaluasi desain dilakukan setelah desain selesai dibuat.
- Dalam pendekatan modern (khususnya **Agile dan DevOps**), evaluasi dilakukan **secara iteratif dan inkremental**, bersamaan dengan setiap *sprint* atau *iteration*.
- Artinya, setiap kali desain diperbarui atau diadaptasi, dilakukan *mini-review* untuk menilai dampaknya terhadap kualitas sistem.
- *Tujuan:* menjaga agar kualitas desain tidak menurun seiring perubahan kebutuhan dan fitur baru.

2. Fokus pada atribut kualitas (Quality Attributes)

Pressman & Maxim menekankan bahwa evaluasi desain modern tidak hanya melihat kelengkapan fungsional, tetapi juga atribut kualitas arsitektur yang memengaruhi kinerja jangka panjang sistem:

Atribut Kualitas	**Penjelasan**
Performance	Seberapa efisien sistem dalam memproses data dan merespons permintaan pengguna.
Scalability	Kemampuan sistem untuk bertumbuh dan menangani peningkatan beban tanpa menurunkan performa.
Modifiability	Kemudahan melakukan perubahan atau penambahan fitur tanpa memengaruhi bagian lain secara signifikan.
Security	Perlindungan terhadap akses tidak sah dan kerentanan desain.

3. Diterapkan melalui Agile Iteration dan Architectural Review

- Dalam praktik *Agile*, evaluasi dilakukan setiap akhir *sprint* sebagai bagian dari *Sprint Review* dan *Retrospective*.
- Tim desain dan pengembang melakukan **Architectural Review Meeting**, yaitu sesi evaluasi untuk memastikan bahwa perubahan kode atau arsitektur masih menjaga integritas desain utama.
- Teknik yang umum digunakan:
  - *Scenario-based evaluation* (evaluasi melalui skenario penggunaan nyata)
  - *ATAM – Architecture Trade-off Analysis Method*
  - *QAW – Quality Attribute Workshop*
- *Hasil yang diharapkan:* desain yang **adaptif, terukur, dan konsisten dengan prinsip kualitas perangkat lunak modern.**

KNOWLEDGE & SOFTWARE ENGINEERING

Teknik Validasi Desain Modern

- Scenario-Based Evaluation: Menguji desain dengan skenario nyata.
- Quality Attribute Workshops (QAW): Diskusi atribut kualitas.
- Architecture Trade-off Analysis Method (ATAM): Mengevaluasi kompromi performa vs maintainability.

1. Scenario-Based Evaluation

- Metode ini menggunakan **skenario penggunaan nyata** untuk menilai apakah rancangan mampu menangani kondisi yang mungkin terjadi di dunia nyata.
- Tim pengembang dan stakeholder membuat daftar skenario seperti:
  - "Bagaimana sistem bereaksi jika jumlah pengguna naik 100 kali lipat?"
  - "Apa yang terjadi jika satu server gagal?"
  - "Bagaimana performa antarmuka saat koneksi lambat?"
- Setiap skenario diuji terhadap desain arsitektur untuk menemukan potensi kelemahan atau risiko.
- *Tujuan:* mengukur ketahanan (*robustness*) dan fleksibilitas desain menghadapi perubahan lingkungan atau kebutuhan.

2. Quality Attribute Workshops (QAW)

- **QAW** adalah teknik kolaboratif di mana tim pengembang, arsitek, dan pengguna berdiskusi tentang **atribut kualitas sistem**.
- Prosesnya melibatkan:
  - Identifikasi atribut kualitas (misalnya: *usability, performance, modifiability*).
  - Penyusunan *utility tree* untuk memprioritaskan atribut yang paling penting.
  - Pembuatan skenario evaluasi berdasarkan atribut prioritas.
- Hasilnya digunakan untuk memperbaiki arsitektur dan memastikan desain mendukung kebutuhan kualitas sistem.
- *Tujuan:* menyelaraskan persepsi semua pihak tentang standar kualitas yang ingin dicapai.

3. Architecture Trade-off Analysis Method (ATAM)

- **ATAM** adalah metode formal untuk **mengevaluasi trade-off** antar atribut kualitas — misalnya antara *performance* dan *maintainability* atau antara *security* dan *usability*.
- Prosesnya terdiri dari:
  - Mengidentifikasi atribut kualitas dan skenario utama.
  - Mengevaluasi dampak keputusan desain terhadap tiap atribut.
  - Menentukan *trade-off points*, yaitu area di mana peningkatan satu atribut dapat menurunkan atribut lain.
- Contoh: penggunaan enkripsi tinggi meningkatkan keamanan tetapi menurunkan performa.
- *Tujuan:* membantu tim membuat keputusan desain yang seimbang dan berdasar analisis risiko.

Metrik Evaluasi Desain (Pressman, 2020)

- Structural Metrics: Coupling, Cohesion, Complexity.
- OO Metrics: RFC, WMC, NOC.
- Agile Metrics: Technical debt, simplicity, design change rate.
- Tujuan: membuat evaluasi desain terukur dan obyektif.

1. Structural Metrics (Metrik Struktural)

- Digunakan untuk menilai **kualitas arsitektur sistem dan relasi antar komponen**. Tiga metrik utama:
- **Coupling** → Mengukur tingkat ketergantungan antar modul.
  - Semakin rendah coupling, semakin mudah sistem dipelihara.
  - Contoh: modul A tidak boleh terlalu sering mengakses variabel modul B.
- **Cohesion** → Mengukur sejauh mana elemen dalam satu modul berfokus pada satu fungsi tunggal.
  - Semakin tinggi cohesion, semakin jelas tanggung jawab modul.
- **Complexity (Kompleksitas)** → Mengukur kerumitan logika program atau struktur kontrol.
  - Umumnya diukur dengan **McCabe's Cyclomatic Complexity (V(G))**, yaitu jumlah jalur independen dalam kode.
  - Kompleksitas tinggi menandakan potensi kesalahan dan kesulitan pemeliharaan.
- *Tujuan:* menjaga agar arsitektur tetap sederhana, modular, dan mudah dikelola.

2. Object-Oriented Metrics (OO Metrics)

- Digunakan pada sistem berbasis **pemrograman berorientasi objek (OOP)**. Beberapa metrik penting yang dijelaskan oleh Pressman:
- **RFC (Response for a Class)** → Jumlah metode yang dapat dipanggil oleh sebuah kelas.
  - Semakin banyak metode yang dapat dipanggil, semakin kompleks perilaku kelas tersebut.
- **WMC (Weighted Methods per Class)** → Total kompleksitas metode dalam satu kelas.
  - Nilai tinggi menunjukkan kelas terlalu kompleks dan sulit diuji.
- **NOC (Number of Children)** → Jumlah subclass yang diwarisi dari suatu kelas induk.
  - Nilai besar bisa berarti kelas dasar memiliki tanggung jawab yang luas (baik untuk reusability, tapi berisiko kompleks).
- *Tujuan:* mengontrol kompleksitas desain berbasis objek agar mudah diuji, diperluas, dan dipelihara.

3. Agile Metrics

- Dalam konteks **pengembangan iteratif modern (Agile)**, Pressman memperkenalkan metrik baru untuk mengukur **kualitas dan stabilitas desain selama perubahan cepat**:
- **Technical Debt** → Jumlah "utang teknis" akibat keputusan desain jangka pendek (misalnya solusi cepat tapi tidak efisien).
- **Simplicity** → Seberapa sederhana struktur desain, dilihat dari ukuran diagram, jumlah dependensi, atau waktu adaptasi tim.
- **Design Change Rate** → Frekuensi perubahan desain antar *sprint*; semakin tinggi nilainya, semakin besar kemungkinan adanya masalah pada arsitektur awal.
- *Tujuan:* memastikan desain tetap stabil dan berkualitas meskipun terus berubah dalam siklus iterasi Agile.

Integrasi dengan Quality Assurance

- Hasil evaluasi desain masuk dalam SQA Plan.
- Mendukung audit kualitas, defect tracking, dan perbaikan proses.
- Membentuk baseline kualitas untuk tahap implementasi dan testing.

1. Hasil evaluasi desain masuk dalam SQA Plan

- Semua hasil penilaian desain (temuan, rekomendasi, dan perbaikan) harus **terdokumentasi** dan **dimasukkan ke dalam SQA Plan** (*Software Quality Assurance Plan*).
- Dokumen ini berfungsi sebagai **rencana pengendalian mutu**, yang menjelaskan:
  - Standar kualitas yang digunakan (misalnya ISO/IEC 25010, CMMI).
  - Prosedur review dan audit desain.
  - Tanggung jawab tim dalam proses evaluasi dan perbaikan.
- Dengan memasukkan hasil evaluasi ke dalam SQA Plan, organisasi memastikan bahwa kualitas desain **dimonitor dan dikendalikan secara sistematis.**
- *Tujuan:* agar semua keputusan desain dapat ditelusuri (*traceable*) dan terdokumentasi dalam sistem manajemen mutu.

2. Mendukung audit kualitas, defect tracking, dan perbaikan proses

- Evaluasi desain menyediakan **data awal** untuk audit kualitas — misalnya daftar kesalahan desain, tingkat modularitas, atau hasil pengujian prototipe.
- Informasi ini digunakan dalam:
  - **Quality Audit:** pemeriksaan periodik terhadap kepatuhan terhadap standar desain.
  - **Defect Tracking:** pencatatan dan pelacakan kesalahan atau kelemahan yang ditemukan pada tahap desain.
  - **Process Improvement:** perbaikan prosedur desain agar kesalahan tidak terulang pada proyek berikutnya.
- *Contoh:* jika desain sering gagal di aspek *modifiability*, maka proses desain diubah agar lebih fokus pada prinsip *low coupling, high cohesion*.

3. Membentuk baseline kualitas untuk tahap implementasi dan testing

- Hasil evaluasi desain berfungsi sebagai **baseline kualitas** — yaitu standar pembanding ketika sistem masuk tahap implementasi dan pengujian.
- Baseline ini mencakup:
  - Standar arsitektur (struktur modul, interface, dependensi).
  - Toleransi kompleksitas atau performa.
  - Kriteria penerimaan (acceptance criteria).
- Saat implementasi atau testing, sistem akan dibandingkan dengan baseline ini untuk memastikan tidak terjadi *deviation* dari rancangan yang sudah diverifikasi.
- *Tujuan:* menjaga konsistensi antara desain, kode, dan hasil uji agar kualitas sistem tetap terjamin.

Checklist Evaluasi Desain

- Apakah semua modul terhubung dengan benar?
- Apakah alur data sesuai dengan spesifikasi?
- Apakah antarmuka memperhatikan usability?
- Apakah desain dapat di-maintain dengan mudah?

1. "Apakah semua modul terhubung dengan benar?"

- Pertanyaan ini menilai **integritas arsitektur sistem**.
- Modul-modul dalam sistem harus berinteraksi sesuai dengan rancangan arsitektur dan tidak saling bergantung secara berlebihan.
- Fokus utama:
  - Hubungan antar modul (interface, data flow, dependencies) sesuai spesifikasi.
  - Tidak ada *circular dependency* yang menyebabkan kesulitan saat kompilasi atau pemeliharaan.
- *Tujuan:* memastikan struktur sistem modular, terintegrasi dengan baik, dan mudah dikembangkan.

KNOWLEDGE & SOFTWARE ENGINEERING

2. "Apakah alur data sesuai dengan spesifikasi?"

- Mengevaluasi **kesesuaian aliran data (data flow)** antara input, proses, dan output dengan dokumen *Software Requirements Specification (SRS)*.
- Aspek yang diperiksa:
  - Aliran data di *Data Flow Diagram (DFD)* atau *Sequence Diagram*.
  - Format dan jenis data yang digunakan antar modul.
  - Validasi bahwa tidak ada data hilang, ganda, atau salah arah.
- *Tujuan:* memastikan sistem memproses dan mentransfer data secara konsisten dan akurat.

KNOWLEDGE & SOFTWARE ENGINEERING

3. "Apakah antarmuka memperhatikan usability?"

- Menilai aspek **kemudahan penggunaan (usability)** pada rancangan *User Interface (UI)* dan *User Experience (UX)*.
- Elemen yang diperhatikan:
  - Konsistensi tata letak dan ikon.
  - Navigasi mudah dan logis.
  - Desain mendukung efisiensi dan meminimalkan kesalahan pengguna.
- Biasanya diuji melalui *prototype evaluation* atau *heuristic review*.
- *Tujuan:* memastikan rancangan antarmuka ramah pengguna dan mendukung produktivitas.

KNOWLEDGE & SOFTWARE ENGINEERING

4. "Apakah desain dapat di-maintain dengan mudah?"

- Fokus pada **maintainability**, yaitu kemampuan sistem untuk diperbaiki, diperbarui, atau dikembangkan tanpa menimbulkan dampak besar.
- Faktor yang dinilai:
  - Struktur desain modular dan reusable.
  - Dokumentasi desain lengkap dan mudah dipahami.
  - Penggunaan pola desain (design patterns) yang tepat.
- *Tujuan*: menjamin sistem tidak hanya berfungsi saat ini, tetapi juga mudah dirawat di masa depan.

Penutup

- Evaluasi bukan mencari kesalahan, tetapi peluang perbaikan.
- Gunakan metrik dan prinsip Pressman (2010, 2020) untuk validasi objektif.

- "Quality means doing it right when no one is looking." – Henry Ford